

# 树状展开执行内核

一种让 Agent「先理解目的、再分形展开」的执行结构 —— 原理、实现与可复现的实测

可运行代码：`tree-kernel.mjs`（单文件·零依赖·Node 18+） | 本报告中的代码与运行输出均为实测结果，可直接复现

## 1. 一句话

执行不是待办清单，而是一次从目的出发的分形展开。

每个节点都是一次「从 A 到 B」：A 是委托人的原话，B 是我理解的目的。要达成 B，就把它拆成几个更小的 B；每个小 B 再拆.....直到「成败可判定」且「一口气能做完」——那就是叶子，直接做。

由此得到一条贯穿全树的性质：**每一层都是同一种循环**。根节点与最深的叶子，结构完全一样，只是抽象层级不同。整个执行过程因此可以写成同一段代码，递归地作用在每个节点上。

## 2. 它要解决的问题

让模型执行任务时，最常见的失效不是「不会做」，而是三种**过程失控**：

| 症状     | 表现                             | 根因                           |
|--------|--------------------------------|------------------------------|
| 直接开干   | 拿到一句话就开始调工具，读到一半才发现理解偏了        | 「理解目的」只发生在模型脑内，没有任何环节强制它留下结果 |
| 假完成    | 任务标成完成，实际有切面没做；或子任务没做完，父任务先标完成 | 完成状态靠「声明」，不靠「证据」             |
| 推不动还硬推 | 同一环节连续失败，仍反复重试同一条路             | 没有「连续失败」这个信号被摆到眼前            |

这三点都不是能力问题，而是**结构缺失**。内核的做法不是写更多提示词去提醒模型，而是让机制在动作发生之前拦住它——**不靠自觉，拦住直到表态**。

## 3. 核心模型：A → B 分形

树上的每个节点只装一件事：**B（这一步要到达什么）**。其余字段都是机制用的状态位。

| 字段     | 含义                            |
|--------|-------------------------------|
| id     | 节点标识，用于引用                     |
| parent | 父节点。为空 = 树的根（一个新任务的起点）        |
| b      | <b>唯一真相源</b> ：本节点要到达的目的       |
| from   | 仅根节点：这句 B 是从委托人哪句话读出来的（A 的留痕） |
| done   | 这个节点被验收过                      |
| hold   | 待确认：理解没把握时先挂起，机制锁住动世界的动作      |
| stale  | 祖先改过目的 → 按旧目的做的一切作废，待复核       |
| review | 根收尾时的复核结论                     |
| fails  | （树级）连续失败计数——驱动变数门；改计划即清零      |

三个设计要点：

- 只有 B，没有「怎么做」**。怎么拆、拆几层，全归模型判断——机制不规定路径，只规定「路径必须被写出来」。
- 父节点升格为目的**。给它挂一个子节点时，它自己就变成目的而非动作，于是同构成立。
- why 链始终可见**。任何节点都能向上追溯到根，因此最深的叶子上干活时，祖先的目的也摆在眼前——不会为了完成一个小步骤而背离大目的。

## 4. 达成是算出来的

这是整个内核最要紧的一条。**完成状态不是被标记出来的，是被计算出来的：**

叶子节点：达成 = 自己 done  
有子的节点：达成 = 自己 done 且 所有子节点都达成

一趟自底向上的遍历即可算出全树状态。它带来两个直接后果：

- **手标父节点无效。**只要还有子节点没达成，标记父节点完成会被机制拒绝并列出未完成项。
- **父节点的「done」含义变了。**它不是「我做完了」，而是「我看过子结果、照目的原文验收过」——验收本身是一次动作，不是自动发生的事。

计算是非递归的（广度定序 + 逆序回推），所以极深的链和环状脏数据都不会爆栈；只连「父节点真实存在」的边，父 id 缺失的孤儿节点会被单独标出为「不可达」，而不是静默地冒充成「还有活」。

## 5. 机制逼思考：三道门

门不是提示词，是**动作发生前的拦截**。每道门都只拦「跳过思考」这件事本身，不评判思考的质量。

### ① 规划门 —— 根已登记但没写路径

登记了一个根节点、却还没拆任何步骤时，机制拦下一切「动手」：写文件、跑命令，**连读文件和搜网页也拦**。理由：摆一个空壳目的就开始盲目探索，是执行跑偏最隐蔽的形式。出口只有一个——先把路径写出来（哪怕只有一步）。

### ② 变数门 —— 连续失败两次

连续失败 2 次且中间没有成功，说明「推不动了」。此时拦下一次动世界的动作，逼执行者先做一件事：要么改计划（`edit / prune`），要么写明为什么计划不变还要重试。改动计划会让失败链清零（已对变数表了态），拦截随之解除。机制只看看得见「连续失败」这个机械信号，分不清「同一环卡住」与「两件无关的事各失败一次」——分辨交给模型。

### ③ 收尾门 —— 根收尾必须交复核结论

根节点收尾时，机制把对账材料摆出来（原始委托、计划清单、实际发生过的动作、失败记录），要求写一段复核结论才能完成。机制只保证「你看过材料再表态」，不判结论对错。

三道门的共同口径：**只逼表态，不判对错**。机制负责让必要的思考必然发生，判断权始终留在模型手里。

## 6. 状态管理的三个选择

| 选择       | 做法                                                  | 理由                               |
|----------|-----------------------------------------------------|----------------------------------|
| 改目的即级联作废 | <code>edit</code> 改写某节点的 B 时，其全部后代被标为 stale、done 清空 | 按旧目的做的成果若继续算数，脏的「已完成」会沿树向上传染成假达成 |
| 剪枝留墓志铭   | <code>prune</code> 删除子树，但把被砍的路线与原因记进 graves         | 被否定的路要留痕，否则同一个错路会重新长出来           |
| 清树前先存底   | <code>reset</code> 前把整棵树写入 <code>archive/</code>    | 「可回溯」必须由机制保证，不能靠记得手动备份           |

## 7. 完整内核代码

单文件、零依赖。保存为 `tree-kernel.mjs`，用 Node 18+ 直接运行。本报告以下代码与后一节运行输出，取自同一份文件的同一次执行。

```
#!/usr/bin/env node
/**
 * tree-kernel.mjs - 树状展开执行内核（精简版）
 *
 * 一个「执行不是待办清单，而是从目的出发的分形展开」的最小可运行内核。
 * 零依赖，Node 18+，单文件。
 *
 * 原理一句话：执行永远是从 A 到 B。
 *   A = 委托人的原话（from），B = 我理解的目的（b）。
 * 要达成 B，就拆成几个小 B：每个小 B 再拆.....直到
 * 「成败可判定」且「一口气能做完」= 叶子，直接做。
 * 每层同构：每个节点下面又是一层同样的循环。
 *
 * 达成是算出来的，不是标出来的：
 * 叶子看自己（done）：有子的节点看「所有子都达成 + 自己那次验收」。
 * 所以手标一个父节点 done 是无效的——机制会拦。
 *
 * 用法：
 * node tree-kernel.mjs begin --b "... " [--from "..."] [--parent <id>] [--hold]
 * node tree-kernel.mjs done --node <id> [--review "..."]      # 根收尾必须带 review
 * node tree-kernel.mjs edit --node <id> --b "... "            # 改目的 → 级联作废后代
 * node tree-kernel.mjs prune --node <id> [--why "..."]         # 剪枝，留墓志铭
 * node tree-kernel.mjs confirm --node <id>                    # 解除 hold
 * node tree-kernel.mjs fail --why "... "                      # 记一次失败（连续 2 次关变数门）
 * node tree-kernel.mjs view | reset | guard | demo
 * 全局选项: --state <文件>（默认 ./tree-state.json）
 */

import * as fs from "node:fs";
import * as path from "node:path";

/* ----- 1. 数据：节点与树 ----- */

const EMPTY = { version: 1, nodes: [], graves: [], fails: 0, updatedAt: "" };

/** 节点 = 一个「从 A 到 B」的目的。b 是唯一真相源，其余字段是机制用的状态位。 */
// { id, parent, b, from, done, hold, stale, review }

/* ----- 2. 计算：一趟算完全树（非递归） ----- */

/** 只连「父真实存在」的边。父 id 缺失 = 孤儿 → 不可达 → 不算活。
 * 否则会冒充「还有活」静默放行，而视图里又看不见它。 */
function analyze(nodes) {
  const byId = new Map(nodes.map((n) => [n.id, n]));
  const kids = new Map();
  for (const n of nodes) {
    const p = n.parent ? byId.get(n.parent) : undefined;
    if (p) kids.set(p.id, [...(kids.get(p.id) ?? []), n]);
  }
  const kidsOf = (n) => kids.get(n.id) ?? [];
  const roots = nodes.filter((n) => !n.parent);

  // 广度优先定序 + 深度：访问过就跳过 → 环数据不爆栈
  const order = [...roots];
  const depth = new Map(roots.map((r) => [r.id, 0]));
  for (let i = 0; i < order.length; i++)
    for (const k of kidsOf(order[i])) {
      if (depth.has(k.id)) continue;
      depth.set(k.id, (depth.get(order[i].id) ?? 0) + 1);
      order.push(k);
    }

  // 逆序 = 自底向上：叶子看自己，父看「子全达成 + 自己那次验收」
  const ok = new Map();
  for (let i = order.length - 1; i >= 0; i--) {
    const n = order[i], k = kidsOf(n);
    ok.set(n.id, k.length > 0 ? n.done && k.every((c) => ok.get(c.id) === true) : n.done);
  }

  const subtree = (root) => {
    const seen = new Set([root.id]), out = [root];
    for (let i = 0; i < out.length; i++)
      for (const k of kidsOf(out[i])) if (!seen.has(k.id)) { seen.add(k.id); out.push(k); }
    return out;
  };

  /** why 链：根 → ... → 本节点。深叶干活时必须看得见祖先目的。 */
  const chain = (n) => {
    const out = [], seen = new Set();
    for (let cur = n; cur && !seen.has(cur.id); cur = cur.parent ? byId.get(cur.parent) : undefined) {
      seen.add(cur.id); out.push(cur);
    }
    return out.reverse();
  };

  const leaves = order.filter((n) => !kidsOf(n).length && !n.done);
  const awaiting = order.filter((n) => kidsOf(n).length > 0 && !n.done && kidsOf(n).every((c) => ok.get(c.id)));
  return {
    roots, order, ok, depth, kidsOf, subtree, chain, leaves, awaiting,
    held: order.filter((n) => n.hold && !n.done),
    stale: order.filter((n) => n.stale && !n.done),
    pending: [...leaves, ...awaiting],
    unreachable: nodes.length - order.length,
  };
}
```

```
/* ----- 3. 三道门：机制逼思考发生 ----- */
```

```
/** 规划门：根已登记但没写路径（光杆根）= 还没想清楚要读什么、做什么 → 除树动作外全拦。  
* 拦的是「动手」这件事本身，含读文件/搜网页/只读命令—摆个空根就能盲目探索，是旧版的病。 */  
function planGate(state) {  
  const a = analyze(state.nodes);  
  const bare = a.roots.filter((r) => !r.done && !r.hold && !a.kidsOf(r).length);  
  if (!bare.length) return null;  
  return 🧑 规划门：根已登记但没写路径—先拆步骤（begin --parent <根id> --b ...），再动手。\\n"  
    + bare.map((r) => ` 根 ${r.id}: ${r.b}` ).join("\\n");  
}  
  
/** 收尾门：根的 done 必须带复核结论—先看对账材料（原话/计划/实际行动），再表态。  
* 只拦「表态没交」这件事，不判结论质量。 */  
function closingGate(state, ids, review) {  
  if (review && review.trim()) return null;  
  const a = analyze(state.nodes);  
  const roots = ids.filter((id) => {  
    const n = state.nodes.find((x) => x.id === id);  
    return !!n && !n.parent && !n.done;  
  });  
  if (!roots.length) return null;  
  return 🧑 收尾门：根收尾要带复核结论—照目的原文过一遍（漏了什么切面/哪些动作没验证/有无越界）。\\n"  
    + roots.map((id) => {  
      const n = state.nodes.find((x) => x.id === id), kids = a.kidsOf(n);  
      return ` 根 ${id}: ${n.b}\\n 原始委托: ${rootFrom(state, n) ?? "（未记）"}\\n`  
        + ` 计划: ${kids.length ? kids.map((c) => `${a.ok.get(c.id) ? "√" : "X"}${c.b}` ).join(" / ") : "单步"}`;  
    }).join("\\n")  
    + "\\n 出口: done --node ${roots[0]} --review "复核结论";  
}  
  
/** 变数门：连续失败 ≥2 且中间无成功 = 「推不动了」的机械信号 → 拦下一次动世界，  
* 逼先调计划（edit/prune）或写明为什么不变。机制只看得见「连续失败」这个信号。 */  
function varianceGate(state) {  
  const fails = state.fails ?? 0;  
  if (fails < 2) return null;  
  return 🧑 变数门：连续失败 ${fails} 次、中间无成功—先调计划再继续。\\n`  
    + ` · 计划错了 → edit --node <id> --b "新做法" 或 prune --node <id> --why "为什么砍"\\n`  
    + ` · 只是重试 → 写明为什么不变，再动手`;  
}  
  
/** 全部拦截合并：变数门优先于规划门（连续失败且连路径都没写时，先补路径这个更靠前的缺口）。 */  
function guard(state) {  
  return varianceGate(state) ?? planGate(state);  
}
```

```
function rootFrom(state, n) {  
  for (const anc of analyze(state.nodes).chain(n)) if (anc.from) return anc.from;  
  return undefined;  
}
```

```
/* ----- 4. 视图与动作 ----- */
```

```
const clip = (s, n = 60) => (s.length > n ? s.slice(0, n) + "..." : s);  
  
function renderAll(state) {  
  if (!state.nodes.length) return "空树。begin 登记根节点。";  
  const a = analyze(state.nodes);  
  const mark = (n) => (n.hold && !n.done ? "||" : a.ok.get(n.id) ? "√"  
    : a.awaiting.includes(n) ? "□" : n.stale ? "⊗" : "...");  
  const deepest = a.order.reduce((m, n) => Math.max(m, a.depth.get(n.id) ?? 0), 0);  
  return [  
    `树: ${a.order.length} 节点 · 最深 ${deepest} 层${a.roots.length > 1 ? ` · ${a.roots.length} 棵` : ""}`,  
    ...a.order.flatMap((n) => {  
      const ind = " ".repeat(a.depth.get(n.id) ?? 0);  
      return [` ${ind}${mark(n)} [${n.id}] ${n.b}`,  
        ...(n.from ? [` ${ind} 原话: ${n.from}`] : []),  
        ...(n.review ? [` ${ind} 复核: ${clip(n.review, 100)}`] : [])];  
    })],  
    ...[a.unreachable ? [🚧 a.unreachable 个节点不可达（父 id 缺失）—prune 或 reset 清理] : []],  
    ...[a.stale.length ? [⊗ 待复核 ${a.stale.length} 个（祖先改过目的）] : []],  
    ...[state.graves.length ? [🪦 已放弃 ${state.graves.length} 条路线（防止错路重长）] : []],  
  ].join("\\n");  
}  
  
/** 每轮摆在眼前的那一小块：根 + 原话 + 待确认 + 待验收 + 待办的活（带 why 链）。 */  
function stateBlock(state) {  
  if (!state.nodes.length) return "【当前树】\\n空树—begin 开树。\\n【/当前树】";  
  const a = analyze(state.nodes), pad = " ", lines = [];  
  for (const r of a.roots) {  
    const tree = a.subtree(r).filter((n) => n.id !== r.id);  
    const settled = tree.length > 0 && tree.every((n) => n.done) && !a.awaiting.includes(r);  
    if (settled) { lines.push(`√ ${r.b}（已完成）`); continue; }  
    lines.push(`根: ${r.hold && !r.done ? "|| 待确认 · " : ""}${r.b}`);  
    if (r.from) lines.push(` ${pad} 原话: ${r.from}`);  
    if (!r.done && !a.kidsOf(r).length) lines.push(` ${pad} 未写路径（规划门）：动手前先拆步骤`);  
    lines.push(...tree.filter((n) => n.hold && !n.done).map((n) => ` ${pad} || 待确认: ${n.b}`));  
    if (a.awaiting.includes(r)) lines.push(` ${pad} 待验收（子都达成了）：照目的原文过一遍 → done --review "..."`);  
    lines.push(...tree.filter((n) => a.awaiting.includes(n)).map((n) => ` ${pad} 待验收: ${n.b}`));  
    lines.push(...tree.filter((n) => !n.done && !n.stale && !n.hold && !a.kidsOf(n).length).map((n) => {  
      const c = a.chain(n).slice(0, -1);  
      return ` ${pad}· ${c.length ? c.map((x) => clip(x.b, 22)).join(" > ") + " > " : ""}${clip(n.b, 80)}`;  
    }));  
    lines.push(...tree.filter((n) => n.stale && !n.done).map((n) => ` ${pad} ⊗ 待复核: ${clip(n.b)}`));  
  }  
  if (!a.pending.length && !a.held.length) lines.push("活都做完了—这个任务结束，新任务 reset 清树。");  
  return `【当前树】\\n${lines.join("\\n")}\\n【/当前树】`;  
}
```

```
function nextId(nodes) {  
  let i = nodes.length + 1;
```

```

const used = new Set(nodes.map((n) => n.id));
while (used.has("n" + i)) i++;
return "n" + i;
}

/** 六个动作。每个都返回一段人类可读的回执——动作与对账都落在同一处。 */
const actions = {
  begin(state, p) {
    const b = String(p.b ?? "").trim();
    if (!b) return { err: "begin 需 --b (这一步要到达什么)" };
    const parent = p.parent?.trim() || null;
    if (parent && !state.nodes.find((x) => x.id === parent)) return { err: `父节点 ${parent} 不存在` };
    const id = nextId(state.nodes);
    const n = { id, parent, b, from: parent ? undefined : (p.from?.trim() || undefined), done: false, hold: !!p.hold };
    const nodes = state.nodes.map((x) => (x.id === parent ? { ...x, done: false, stale: undefined } : x));
    return {
      state: { ...state, nodes: [...nodes, n], fails: 0, updatedAt: new Date().toISOString() },
      text: `${id} 已登记。${parent ? ` (${parent} 的一个切面, ${parent} 升格为目的)` : ""}`,
    };
  },

  done(state, p) {
    const id = p.node?.trim();
    const n = state.nodes.find((x) => x.id === id);
    if (!n) return { err: `节点 ${id} 不存在` };
    const a = analyze(state.nodes);
    if (!a.depth.has(id)) return { err: `节点 ${id} 不可达, 先修复或 prune` };
    if (a.chain(n).some((x) => x.hold && !x.done)) return { err: `节点 ${id} 或其祖先尚待 confirm, 不能 done` };
    const notYet = a.kidsOf(n).filter((c) => !a.ok.get(c.id));
    if (notYet.length) return { err: `节点 ${id} 还有 ${notYet.length} 个子没达成, 不能手标 (达成是算出来的): \n`
      + notYet.map((c) => `  · ${clip(c.b)} `).join("\n") };
    const gate = closingGate(state, [id], p.review);
    if (gate) return { err: gate };
    const review = String(p.review ?? "").trim();
    return {
      state: { ...state, nodes: state.nodes.map((x) => (x.id === id
        ? { ...x, done: true, stale: undefined, review: (!x.parent && review) ? review : x.review } : x)),
        updatedAt: new Date().toISOString() },
      text: `${id} 已标记完成${a.kidsOf(n).length ? " (子结构已完成, 目的须证据复核)" : ""}。 \n`
        + `验收锚: ${n.b}\n原始委托: ${rootFrom(state, n) ?? " (未记)"}`,
    };
  },

  /** 改目的 = 后代按旧目的做的一切都不算数 (级联作废) , 否则脏 done 会沿树上传染成假达成。 */
  edit(state, p) {
    const id = p.node?.trim(), n = state.nodes.find((x) => x.id === id);
    if (!n) return { err: `节点 ${id} 不存在` };
    if (!p.b?.trim()) return { err: "edit 需 --b 给出新的目的" };
    const doomed = analyze(state.nodes).subtree(n).filter((x) => x.id !== n.id);
    const ids = new Set(doomed.map((d) => d.id));
    const undone = doomed.filter((d) => d.done).length;
    return {
      state: { ...state, nodes: state.nodes.map((x) => x.id === id
        ? { ...x, b: p.b.trim(), done: false, stale: undefined, review: undefined }
        : ids.has(x.id) ? { ...x, stale: true, done: false, review: undefined } : x),
        fails: 0, updatedAt: new Date().toISOString() },
      text: `${id} 已改写${n.done ? " (原达成作废)" : ""}。 \n`
        + `(doomed.length ? ` 级联作废 ${doomed.length} 个后代${undone ? ` (${undone} 个原已达成)` : ""}, 按新目的复核。` : ""`,
    };
  },

  /** 剪枝 = 删节点及其子树, 留墓志铭——砍掉的路线 (含砍因) 进记忆, 防错路重长。 */
  prune(state, p) {
    const id = p.node?.trim(), n = state.nodes.find((x) => x.id === id);
    if (!n) return { err: `节点 ${id} 不存在` };
    const gone = new Set(analyze(state.nodes).subtree(n).map((d) => d.id));
    const doneAmong = state.nodes.filter((x) => gone.has(x.id) && x.done).length;
    const g = { t: new Date().toISOString().slice(0, 16).replace("T", " "), b: n.b, why: p.why?.trim() || undefined };
    return {
      state: { ...state, nodes: state.nodes.filter((x) => !gone.has(x.id)),
        graves: [...state.graves, g].slice(-200), fails: 0, updatedAt: new Date().toISOString() },
      text: `已剪枝 ${gone.size} 个节点${doneAmong ? ` (含 ${doneAmong} 个已达成—作废)` : ""}。 \n`
        + ` 墓志铭: ${g.b}${g.why ? ` (${g.why})` : ""}`,
    };
  },

  confirm(state, p) {
    const id = p.node?.trim();
    const targets = id ? state.nodes.filter((x) => x.id === id && x.hold) : state.nodes.filter((x) => x.hold);
    if (!targets.length) return { err: id ? `节点 ${id} 没有待确认的 hold` : "树上没有待确认的节点" };
    const ids = new Set(targets.map((t) => t.id));
    return {
      state: { ...state, nodes: state.nodes.map((x) => (ids.has(x.id) ? { ...x, hold: undefined } : x)),
        updatedAt: new Date().toISOString() },
      text: `* 已解除 ${targets.length} 处 hold (须有明确授权; 状态动作不产生授权事实)。`,
    };
  },

  /** 记一次「动世界失败」。连续失败 ≥2 → 变数门关闭。改计划 (edit/prune) 或 begin 会清零。 */
  fail(state, p) {
    const why = p.why?.trim();
    if (!why) return { err: "fail 需 --why 说明失败在哪" };
    const fails = (state.fails ?? 0) + 1;
    return {
      state: { ...state, fails, updatedAt: new Date().toISOString() },
      text: `X 已记一次失败 (连续 ${fails} 次)。 ${fails} ≥ 2 ? "\n 变数门已关: 下次动世界前先调计划 (edit/prune)。` : ""`,
    };
  },
};

/* ----- 5. 落盘与命令行 ----- */

```

```

const readState = (file) => {
  try { return JSON.parse(fs.readFileSync(file, "utf8")); } catch { return { ...EMPTY }; }
};
const writeState = (file, s) => {
  fs.mkdirSync(path.dirname(path.resolve(file)), { recursive: true });
  fs.writeFileSync(file, JSON.stringify(s, null, 1));
};
/** 存底: 清树前把整棵树挪进 archive/—「存底」这件事保证发生（可回溯）。 */
function archive(file, s, reason) {
  if (!s.nodes.length && !s.graves.length) return true;
  try {
    const dir = path.join(path.dirname(path.resolve(file)), "archive");
    fs.mkdirSync(dir, { recursive: true });
    const stamp = new Date().toISOString().slice(0, 19).replace(/[:T-]/g, "");
    let f = path.join(dir, `tree-${stamp}.json`);
    for (let i = 2; fs.existsSync(f); i++) f = path.join(dir, `tree-${stamp}-${i}.json`);
    fs.writeFileSync(f, JSON.stringify({ reason, nodes: s.nodes, graves: s.graves }, null, 1));
    return true;
  } catch { return false; }
}

function parseArgs(argv) {
  const [cmd, ...rest] = argv;
  const p = {}, pos = [];
  for (let i = 0; i < rest.length; i++) {
    const a = rest[i];
    if (a.startsWith("--")) {
      const k = a.slice(2);
      if (k === "hold") p.hold = true;
      else p[k] = rest[++i];
    } else pos.push(a);
  }
  return { cmd, p, pos };
}

function main() {
  const { cmd, p, pos } = parseArgs(process.argv.slice(2));
  const file = p.state || "./tree-state.json";
  const s = readState(file);
  const show = (t) => console.log(t + "\n\n" + stateBlock(s));

  if (cmd === "view") return console.log(renderAll(s));
  if (cmd === "guard") return console.log(guard(s) ?? "✅ 无拦截: 可以动世界。");
  if (cmd === "reset") {
    const kept = archive(file, s, "reset 前存底");
    writeState(file, { ...EMPTY, updatedAt: new Date().toISOString() });
    return console.log(kept ? "树已清空（原树存底到 archive/, 可回溯）。" : "树已清空。🔥 存底失败—没有可回溯的底。");
  }
  if (cmd === "demo") return; // demo 由 demo() 处理, 见下
  const act = actions[cmd];
  if (!act) {
    console.log("用法: begin | done | edit | prune | confirm | fail | view | reset | guard | demo\n"
      + " 全局: --state <文件>");
    process.exitCode = 1;
    return;
  }
  const r = act(s, p);
  if (r.err) { console.log("X " + r.err); process.exitCode = 1; return; }
  writeState(file, r.state);
  show(r.text);
}

/* ----- 6. demo: 一次完整生命周期（真实输出） ----- */

function demo() {
  const file = path.join(process.env.TMPDIR || process.env.TEMP || "/tmp", "tree-kernel-demo.json");
  fs.rmSync(file, { force: true });
  fs.rmSync(path.join(path.dirname(file), "archive"), { recursive: true, force: true });
  const log = [];
  const run = (label, argv) => {
    const { cmd, p } = parseArgs(argv);
    const s = readState(file);
    let out;
    if (cmd === "view") out = renderAll(s);
    else if (cmd === "guard") out = guard(s) ?? "✅ 无拦截: 可以动世界。";
    else if (cmd === "reset") {
      const kept = archive(file, s, "reset 前存底");
      writeState(file, { ...EMPTY });
      out = kept ? "树已清空（原树存底到 archive/, 可回溯）。" : "树已清空。🔥 存底失败。";
    } else {
      const r = actions[cmd](s, p);
      if (r.err) out = "X " + r.err;
      else { writeState(file, r.state); out = r.text; }
    }
    log.push(`\n$ node tree-kernel.mjs ${argv.join(" ")}\n${out}`);
    return out;
  };

  run("1 登记根（带着原话）", ["begin", "--b", "把树机制讲清楚并交付一份能跑的证明", "--from", "给我出一份树机制的 PDF 报告"]);
  run("2 规划门: 光杆根时不许动手", ["guard"]);
  run("3 拆两个切面", ["begin", "--parent", "n1", "--b", "写原理说明"]);
  run("4 拆到叶子（一口气能做完）", ["begin", "--parent", "n1", "--b", "写可运行内核代码"]);
  run("5 达成是算出来的: 父还有子没完成", ["done", "--node", "n1", "--review", "想跳过"]);
  run("6 叶子直接做", ["done", "--node", "n2"]);
  run("7 第二片叶子也做", ["done", "--node", "n3"]);
  run("8 子全达成 → 父变待验收", ["view"]);
  run("9 收尾门: 根收尾必须带复核结论", ["done", "--node", "n1"]);
  run("10 交了复核结论, 验收通过", ["done", "--node", "n1", "--review", "照目的原文过: 原理+可跑代码两切面都在, 无越界。"]);
  run("11 第一次失败", ["fail", "--why", "npm 装不上依赖"]);
  run("12 第二次失败 → 变数门关闭", ["fail", "--why", "换了个镜像还是超时"]);
  run("13 推不动了: 动世界被拦", ["guard"]);
  run("14 改计划 → 级联作废 + 失败链清零", ["edit", "--node", "n1", "--b", "换一种讲法: 从达成算法切入"]);
}

```

```
run("15 拦解除，可以继续", ["guard"]);
run("16 错路砍掉，留墓志铭", ["prune", "--node", "n2", "--why", "这条路的写法已过时"]);
run("17 清树前存底", ["reset"]);
console.log(log.join("\n"));
}

/* ----- 入口 ----- */

if (process.argv[2] === "demo") demo();
else main();
```

## 8. 实测运行记录

命令：`node tree-kernel.mjs demo`（下方为该命令的真实输出，未做任何修改）。

它按顺序走完一个完整生命周期，并刻意触发每一次拦截：

1. 登记根节点，并记下原始委托
2. 光杆根 → **规划门拦截**
3. 拆两个切面（父节点随之升格为目的）
4. 试图跳过子节点直接完成父节点 → **被拒绝，并列出未完成项**
5. 逐片叶子完成 → 父节点自动变为「待验收」
6. 根节点无复核结论收尾 → **收尾门拦截**
7. 交出复核结论 → 验收通过
8. 连续两次失败 → **变数门关闭，动世界被拦**
9. 改写目的 → **2 个后代级联作废（含 2 个原已达成）**，失败链随之清零、拦截解除
10. 剪枝 → 留下墓志铭
11. 清树 → 自动存底到 `archive/`

```
$ node tree-kernel.mjs begin --b 把树机制讲清楚并交付一份能跑的证明 --from 给我出一份树机制的 PDF 报告
n1 已登记。

$ node tree-kernel.mjs guard
🕒 规划门：根已登记但没写路径—先拆步骤（begin --parent <根id> --b ...），再动手。
  根 n1：把树机制讲清楚并交付一份能跑的证明

$ node tree-kernel.mjs begin --parent n1 --b 写原理说明
n2 已登记。（n1 的一个切面，n1 升格为目的）

$ node tree-kernel.mjs begin --parent n1 --b 写可运行内核代码
n3 已登记。（n1 的一个切面，n1 升格为目的）

$ node tree-kernel.mjs done --node n1 --review 想跳过
X 节点 n1 还有 2 个子没达成，不能手标（达成是算出来的）：
  · 写原理说明
  · 写可运行内核代码

$ node tree-kernel.mjs done --node n2
n2 已标记完成。
验收锚：写原理说明
原始委托：给我出一份树机制的 PDF 报告

$ node tree-kernel.mjs done --node n3
n3 已标记完成。
验收锚：写可运行内核代码
原始委托：给我出一份树机制的 PDF 报告

$ node tree-kernel.mjs view
树：3 节点 · 最深 1 层
□ [n1] 把树机制讲清楚并交付一份能跑的证明
  ↳ 原话：给我出一份树机制的 PDF 报告
  ✓ [n2] 写原理说明
  ✓ [n3] 写可运行内核代码

$ node tree-kernel.mjs done --node n1
X 🚪 收尾门：根收尾要带复核结论—照目的原文过一遍（漏了什么切面/哪些动作没验证/有无越界）。
  根 n1：把树机制讲清楚并交付一份能跑的证明
  原始委托：给我出一份树机制的 PDF 报告
  计划：✓写原理说明 / ✓写可运行内核代码
  出口：done --node n1 --review "复核结论"

$ node tree-kernel.mjs done --node n1 --review 照目的原文过：原理+可跑代码两切面都在，无越界。
n1 已标记完成（子结构已完成，目的须据证据复核）。
验收锚：把树机制讲清楚并交付一份能跑的证明
原始委托：给我出一份树机制的 PDF 报告

$ node tree-kernel.mjs fail --why npm 装不上依赖
X 已记一次失败（连续 1 次）。

$ node tree-kernel.mjs fail --why 换了个镜像还是超时
X 已记一次失败（连续 2 次）。
🔒 变数门已关：下次动世界前先调计划（edit/prune）。

$ node tree-kernel.mjs guard
🔒 变数门：连续失败 2 次、中间无成功—先调计划再继续。
  · 计划错了 → edit --node <id> --b "新做法" 或 prune --node <id> --why "为什么砍"
  · 只是重试 → 写明为什么不变，再动手

$ node tree-kernel.mjs edit --node n1 --b 换一种讲法：从达成算法切入
n1 已改写（原达成作废）。
🕒 级联作废 2 个后代（2 个原已达成），按新目的复核。

$ node tree-kernel.mjs guard
✅ 无拦截：可以动世界。

$ node tree-kernel.mjs prune --node n2 --why 这条路的写法已过时
已剪枝 1 个节点。
🪦 墓志铭：写原理说明（这条路的写法已过时）
```



## 9. 怎么跑

把代码存成 `tree-kernel.mjs`，无需安装任何依赖：

```
node tree-kernel.mjs demo                # 完整生命周期演示
node tree-kernel.mjs begin --b "要到达什么" --from "原话" # 登记根节点
node tree-kernel.mjs begin --parent n1 --b "第一步"        # 拆一个切面
node tree-kernel.mjs guard                # 查当前是否被拦（规划门 / 变数门）
node tree-kernel.mjs fail --why "失败在哪"                # 记一次失败（连续 2 次关变数门）
node tree-kernel.mjs done --node n2          # 叶子完成
node tree-kernel.mjs done --node n1 --review "复核结论"   # 根收尾（必须带结论）
node tree-kernel.mjs view                  # 看全树
node tree-kernel.mjs reset                 # 清树（自动存底）
```

状态默认存在当前目录的 `.tree-state.json`，可用 `--state <文件>` 指定；每次 `reset` 前的旧树会写入同目录的 `archive/`。

## 10. 边界：这是内核，不是完整系统

本内核只保留「树状展开」这一条主干。作为独立可跑的最小实现，它有意省略了完整系统里的这些部分：

| 完整系统里有                       | 内核里的状态                               |
|------------------------------|--------------------------------------|
| 把门接到 Agent 工具调用循环上（真正拦截工具执行） | 以 <code>guard</code> 子命令显式查询，便于观察与测试 |
| 每轮自动把当前树注入模型上下文              | 由 <code>view</code> / 回执输出呈现         |
| 状态随会话分支持久化与恢复                | 落到本地 JSON 文件                         |
| 对账材料从会话记录自动读取（不靠自述）          | 材料来自树自身记录                            |
| 并发多树、批量操作、注入缓存优化             | 未包含                                  |

被省略的都是「接入层」与「性能层」。**结构本身是完整的**：分形展开、达成计算、三道门、级联作废、墓志铭、存底，这些构成机制的部分一个不少，且行为与完整版一致。

代码：`tree-kernel.mjs`（411 行） · 运行环境：Node v24.15.0 · 本报告中的全部输出均为真实执行结果